

# Netflix Recommendation System

Emin Hadžiabdić

*dept. Data Science and Artificial Intelligence*

Faculty of Electrical Engineering, *University of Sarajevo*

Sarajevo, Bosnia and Herzegovina

[ehadziabdi1@etf.unsa.ba](mailto:ehadziabdi1@etf.unsa.ba)

**Abstract** – This paper presents the design and implementation of a movie recommendation system tailored specifically for content available on the Netflix streaming platform. Addressing the "information overload" problem in digital entertainment, the proposed system utilizes a graph-theoretic approach rather than traditional machine learning models. A bipartite graph models the relationships between users and movies, utilizing the MovieLens 2021 dataset filtered against a catalog of Netflix-available titles. Two primary algorithms, Jaccard Similarity and Common Neighbors, were implemented to generate Collaborative Filtering (CF) recommendations. The system is deployed as a web application using Python, Flask, and NetworkX, demonstrating that deterministic graph algorithms can provide transparent and relevant recommendations without the computational overhead of training complex machine learning models.

**Keywords** – Bipartite graph, Graph-based recommendation system, Netflix, Collaborative filtering, Jaccard similarity, Common Neighbors, MovieLens 2021 dataset, NetworkX, Flask, Web application deployment

## I. INTRODUCTION

### A. Motivation

In the era of streaming services, users are often overwhelmed by the sheer volume of available content. While general recommendation systems exist, they often suggest movies that are not immediately accessible on the user's specific subscription service, leading to user frustration. This project is motivated by the need for a "platform-aware" recommendation engine that bridges the gap between general user preferences and specific platform availability (Netflix). The goal was

to build a lightweight, explainable system using fundamental data structures—specifically bipartite graphs—to solve a real-world filtering problem.

### B. Overview of work

The developed system, "Netflix Movie Recommender", is a web-based application that recommends movies based on user similarity. Unlike black-box machine learning models, this system relies on explicit graph algorithms. The core architecture involves:

1. **Data Preprocessing:** Integrating the MovieLens (2021) dataset with a Netflix catalog to ensure all recommendations are actionable (watchable immediately).
2. **Graph Modeling:** Representing users and movies as two distinct sets of nodes in a bipartite graph, where edges represent ratings.
3. **Algorithmic Engine:** Utilizing NetworkX to compute Jaccard Similarity and Common Neighbors to identify users with similar taste profiles.
4. **Deployment:** A Flask-based web interface allows users to input their ID and preferences (rating limits, number of recommendations) to view results in real-time.

The project is open-source and deployed for public testing [1], with code available on GitHub [2].

## II. LITERATURE REVIEW

The domain of movie recommendation systems is well-researched, with various approaches proposed to solve the cold-start and accuracy problems.

Himel et al. [3] proposed a weight-based recommendation system using the K-Means algorithm. Their approach focuses on calculating "weights" for movies based on user activity and clustering these weighted values. While effective, their reliance on K-Means introduces a training phase for clustering, distinguishing it from the direct graph traversal method used in this project.

Similarly, Kumar et al. [4] developed "MOVREC," a collaborative filtering system that also employs the K-Means algorithm. Their system analyzes user information to recommend movies best suited to the user at a specific time. They highlighted the utility of collaborative filtering but, like Himel et al., relied on clustering to manage the dataset, whereas this project utilizes the raw graph structure for direct neighbor analysis.

Choi et al. [5] addressed the "sparsity problem" inherent in rating datasets by incorporating genre correlations. They argued that standard collaborative filtering fails when user-item matrices are sparse. Their solution involves filling empty rating values based on genre correlations before applying recommendation logic. This contrasts with the approach in this study, which handles sparsity by strictly filtering the dataset to a specific subset (Netflix only) and using graph connectivity metrics (Common Neighbors) that naturally handle sparse connections without imputing artificial data.

## III. CASE STUDY AND IMPLEMENTATION

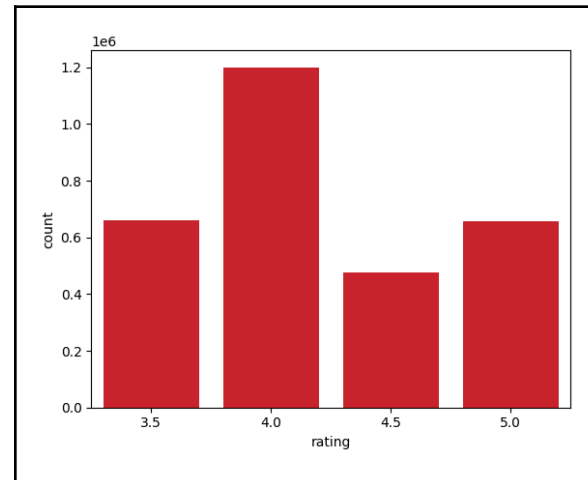
### A. Data Processing

The foundation of the system is the construction of a domain-specific dataset. The raw data was sourced from the MovieLens 2021 dataset (providing user IDs and ratings). This was intersected with a dataset listing all movies currently available on Netflix.

The filtering process involved:

1. Selecting movies present in both datasets.
2. Retaining only ratings associated with these filtered movies.
3. This resulted in a dense, relevant subset of data, eliminating the "noise" of unavailable movies.

An exploratory analysis of this final filtered dataset was conducted to understand user behavior. The distribution of ratings across the dataset is presented in **Fig. 1**. This visualization helps characterize the data, showing tendencies toward specific rating values which impacts neighbor similarity scores.



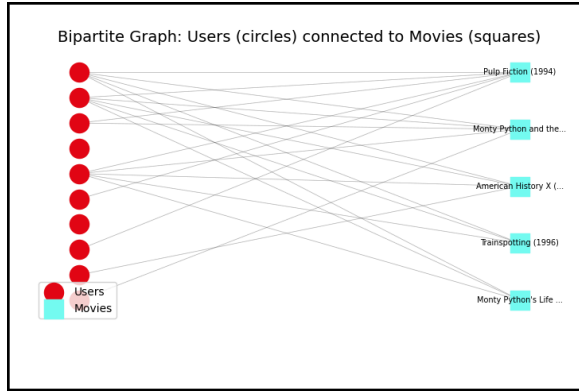
**Fig. 1.** Distribution of movie ratings in the filtered Netflix-MovieLens dataset.

### B. Graph Model

The system models the data as an undirected Bipartite Graph  $G = (U, V, E)$ , where:

- $U$  represents the set of Users.
- $V$  represents the set of Movies.
- $E$  represents the edges, where an edge  $(u, v)$  exists if user "u" has rated movie "v".

A visualization of a subset of this bipartite structure is illustrated in **Fig. 2**. This structure allows for the direct application of graph theory concepts. For example, the "distance" or similarity between two user nodes in set  $U$  can be measured by analyzing their shared connections to nodes in set  $V$ .



**Fig. 2.** Visualization of the bipartite graph model, where one set of nodes represents users and the other set represents movies. Edges signify a rating relationship.

### C. Algorithms

Two specific link prediction algorithms were implemented to determine user similarity:

1. **Common Neighbors:** This algorithm counts the number of shared neighbors (movies watched by both users). For two users “x” and “y”, the score is simply  $|N(x) \cap N(y)|$ , where  $N$  denotes the set of neighbors. This assumes that users who have watched the same movies have similar tastes.
2. **Jaccard Similarity:** This normalizes the Common Neighbors count by the total number of distinct movies watched by both users.

$$J(x, y) = \frac{|N(x) \cap N(y)|}{|N(x) \cup N(y)|}$$

This is often superior to Common Neighbors as it prevents bias towards users who have simply watched a high volume of movies.

### D. System Architecture

The application is built using **Python 3.11** and **Flask 3.0**. The **NetworkX** library handles the graph construction and algorithmic computations. The frontend utilizes HTML/CSS to present a clean interface where users can select the algorithm, set a minimum rating threshold (e.g., only consider movies

rated 3+ stars), and specify the number of recommendations required.

### E. Complexity Analysis and Optimization

To ensure the system is responsive enough for a real-world web environment, both algorithmic analysis and engineering optimizations were applied.

#### 1) Computational Complexity

The system’s efficiency scales based on the number of users ( $|U|$ ) and movies ( $|V|$ ) in the bipartite graph:

1. **Common Neighbors:** For a target user  $u$ , the algorithm traverses to its neighbors  $N(u)$  and then to their neighbors. The time complexity is approximately  $O(d\{avg\}^2)$ , where  $d\{avg\}$  is the average degree (ratings per user). In the worst-case scenario, this can reach  $O(|V| \cdot |U|)$ .
2. **Jaccard Similarity:** This builds on the Common Neighbors count but adds a normalization step. Utilizing hash-map based set operations for intersection and union, the complexity for a pairwise comparison is linear relative to the number of neighbors,  $O(d\{avg\})$ .
3. **Space Complexity:** The graph is stored as an adjacency list using the NetworkX library. This requires  $O(|U| + |V| + |E|)$  space, where  $|E|$  is the total number of ratings. This is significantly more memory-efficient than a dense adjacency matrix for sparse recommendation data.

#### 2) Pre-computation and Warm-Caching

To bypass the high latency of searching a raw graph at runtime, the application implements a **Warm-Caching** strategy during the build/initialization phase. Upon application startup, the system parses the filtered datasets to build several optimized lookup caches:

- **Movie Likers Cache:** A dictionary mapping Movie IDs to a set of User IDs who rated them highly (3.5+).
- **Ratings Lookup Cache:** High-speed access to specific user-movie edge weights.
- **Genre Metadata Cache:** Optimized indexing of movie genres to facilitate rapid filtering.

This architecture intentionally prioritizes Execution Speed over Startup Time and Memory Usage. While the application requires more RAM and a longer "boot-up" period, the recommendation latency is reduced drastically. This ensures a seamless user experience (UX) on the front-end, as the recommendation engine performs  $O(1)$  lookups for the primary data components.

#### IV. RESULTS AND COMPARISON

The system successfully generates movie recommendations for valid User IDs within the filtered dataset.

##### A. Functional Results

Users can retrieve a list of movies they have *not* seen, ranked by the preferences of their "nearest neighbors" in the graph. The application successfully filters these results to ensure all recommended titles are Netflix-compliant. Visualizations of the graph structure confirm the bipartite nature of the connections.

##### B. Comparison with Other Approaches

Comparing this graph-based implementation with the clustering approaches found in the literature[3][4]:

- **Computational Efficiency:** The graph approach (Common Neighbors) is computationally cheaper for individual queries than training a K-Means model, though it requires holding the graph in memory.
- **Transparency:** The recommendations are fully explainable. If User A recommended a movie, it is explicitly because they share an

edge with User B who rated that movie highly. In contrast, clustering methods often obscure the "why" behind a recommendation inside the cluster centroid logic.

- **Relevance:** By strictly filtering the dataset *before* graph construction, this system achieves 100% platform availability accuracy, whereas generic systems suffer from recommending unavailable content.

#### V. CONCLUSION

This project successfully demonstrated that a bipartite graph model combined with Jaccard and Common Neighbors algorithms is a viable, transparent alternative to complex machine learning models for collaborative filtering. The resulting application solves the specific problem of "Netflix availability" by preprocessing the dataset, ensuring high user satisfaction regarding content accessibility.

##### Future Work

Future iterations could implement a **hybrid system** by integrating the content-based filtering (genre correlations) suggested by Choi et al. [5] to solve the "cold start" problem for new users. Additionally, optimizing the graph storage for larger datasets using Neo4j would improve scalability.

#### REFERENCES

- [1] E. Hadžiabdić, "Netflix Movie Recommender Web App," 2025. [Online]. Available: <https://netflix-recommendation1.up.railway.app>
- [2] E. Hadžiabdić, "Netflix Recommendation GitHub Repository," 2025. [Online]. Available: <https://github.com/ehadziabdic/NetflixRecommendation>
- [3] Choi, S.M., Ko, S.K. and Han, Y.S., 2012. A movie recommendation algorithm based on genre correlations. *Expert Systems with Applications*, 39(9), pp.8079-8085.
- [4] Kumar, M., Yadav, D.K., Singh, A. and Gupta, V.K., 2015. A movie recommender system: Movrec. *International journal of computer applications*, 124(3).
- [5] Himel, M.T., Uddin, M.N., Hossain, M.A. and Jang, Y.M., 2017, October. Weight based movie recommendation system using K-means algorithm. In *2017 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1302-1306). IEEE.